DEEP LEARNING COURSE

Data Preprocessing

"Garbage In, Garbage Out"

Teaching Assistant ·

RECITATION 0.17

Data Preprocessing

Aryan Singh Chandel

Course Module Overview

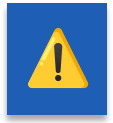
1. Why preprocessing is the most important step in any DL pipeline
2. What 'Garbage In, Garbage Out' really means in practice
3. Audio preprocessing: from raw waveforms to MFCCs
4. Image preprocessing: normalization and augmentation
5. How preprocessing choices encode domain knowledge
6. Why a clean pipeline beats a bigger model

Preprocessing determines whether your model has any chance of learning.

The core principle of deep learning is deceptively simple:

> "Garbage In, Garbage Out" — No matter how sophisticated your architecture, the model can only learn from the patterns that exist in its input. If the input is noisy, inconsistent, or poorly scaled, the model will learn noise.

What this means concretely:



Raw data is not model-ready

Audio files contain tens of thousands of samples per second. Images contain millions of pixel values. Text is just a string of characters. None of this is directly meaningful to a neural network.

Preprocessing transforms data into a learnable representation

We compress, normalize, and restructure the raw input so that patterns the model needs to detect are geometrically accessible to gradient descent.

The pipeline is the first engineering decision you make

Before choosing an optimizer, a loss function, or a number of layers — you decide what your model actually sees. That decision shapes everything

SPEAKER NOTES

Before we touch a single architecture decision, we need to understand what we're feeding the model. Think of preprocessing as translating raw reality into a language the network can read. The best model in the world will fail if it's reading gibberish.

Raw data has three properties that make direct training nearly impossible.

01

High Dimensionality

A single second of CD-quality audio contains 44,100 floating-point numbers. A single 1080p image contains over 6 million values across three colour channels. This is not a problem of storage — it is a problem of learning signal density. The useful pattern is buried inside an enormous space of irrelevant numbers.

02

Inconsistent Scale

One audio recording might have amplitude values between -0.01 and 0.01. Another might range between -100 and 100. One image might have pixel values 0–255, another 0.0–1.0 after a prior conversion. When scales differ across samples, the model's weight gradients are dominated by whichever input has the largest magnitude — completely distorting learning.

03

Noise and Irrelevant Variation

Background hiss, sensor artifacts, lighting differences, camera angle — none of these contain the signal your model is supposed to learn. If you feed raw data directly, your model has no way to distinguish between the meaningful variation (a dog versus a cat) and meaningless variation (the camera that took the photo). It will try to learn both, and fail at both.

SPEAKER NOTES

These are not abstract concerns — they are the specific failure modes you will hit if you skip preprocessing. High dimensionality means the model is searching for a needle in an enormous haystack. Inconsistent scale breaks gradient descent. Noise means the model learns the wrong patterns. Preprocessing addresses all three.

A raw audio waveform contains the right information in the worst possible format.

Consider what a raw waveform actually is:

> A sequence of pressure measurements, typically sampled 44,100 times per second. Each number tells you how much the air pressure deviated from baseline at that exact instant. Nothing more.

Why this is a terrible representation for a neural network:

1. **Time-domain information is redundant and phase-shifted**

   Whether a phoneme starts 2ms earlier or later in the waveform looks completely different numerically, but is semantically identical. The model has to learn this invariance from scratch, wasting enormous capacity.

2. **Frequency relationships are hidden**

   Human speech and music are fundamentally about frequency content — vowels are defined by their formant frequencies, notes by their pitch. None of this is visible in the raw waveform without transformation.

3. **The dimensionality is extreme**

   44,100 inputs per second means a 3-second audio clip is a 132,300-dimensional input. Most phonetic distinctions only require examining 13–40 frequency-band energies over short windows. We are wasting over 99.97% of the representation.

SPEAKER NOTES

The waveform is a faithful record of physical reality, but it is optimized for physics, not for learning. We need to transform it into something that captures the structure humans actually use to distinguish sounds — and that means thinking about frequency, not time samples.

Human hearing already solves this problem — our pipeline is a computational mirror of the cochlea.

What your ear actually does when it hears sound:

Step 1

1

Sound reaches the cochlea

The cochlea is a fluid-filled spiral tube. When sound enters it, different parts of the tube resonate at different frequencies — the base responds to high frequencies, the apex to low frequencies. This is a physical Fourier decomposition happening in your inner ear.

Step 2

2

The basilar membrane applies a mel-scale filter

The cochlea does not space frequencies linearly. It is much more sensitive to differences at low frequencies (200 Hz vs 400 Hz) than at high frequencies (8000 Hz vs 8200 Hz). This is the mel scale — a perceptual, logarithmic spacing of frequency that mirrors what actually matters for speech discrimination.

Step 3

3

The auditory nerve fires compressed feature codes

What gets sent to the brain is not the raw pressure wave — it is a compressed summary: which frequency bands are active, and how strongly. Roughly 30,000 nerve fibres carry this information, a massive reduction from the continuous pressure signal.

SPEAKER NOTES

This is not a coincidence or an arbitrary engineering choice. MFCCs were designed by speech scientists who studied the auditory system and asked: what is the minimal representation that preserves everything a human needs to distinguish phonemes? The answer turned out to map almost perfectly onto cochlear biology.

Computing MFCCs is a four-step pipeline, each step with a specific mathematical justification.

1

STFT — Short-Time Fourier Transform

We cannot apply a Fourier transform to the entire signal at once because speech is non-stationary — the frequency content changes over time. Instead, we slice the signal into short overlapping frames (typically 25ms wide, 10ms hop) and apply an FFT to each frame. This gives us a spectrogram: a 2D map of frequency energy over time.

2

Mel Filterbank

We apply a bank of triangular filters spaced on the mel scale. The mel scale is defined as $m = 2595 \times \log_{10}(1 + f/700)$. This compresses the frequency axis so that low-frequency resolution is high (where speech information is dense) and high-frequency resolution is lower. We typically use 40 filters, reducing the FFT bins dramatically.

3

Log Compression

We take the logarithm of each filterbank energy. This step is motivated by human loudness perception: we perceive loudness on a log scale (decibels). It also makes the feature values more Gaussian-distributed, which helps the downstream model and any normalization we apply later.

4

DCT — Discrete Cosine Transform

The final step applies a DCT to the log filterbank energies. This decorrelates the features (adjacent filterbank outputs are highly correlated because their frequency bands overlap) and compresses them into 13–40 coefficients. These are your MFCCs. The first coefficient captures overall energy; the rest capture the spectral shape.

SPEAKER NOTES

Each step has a precise reason for existing. The STFT handles non-stationarity. The mel filterbank handles perceptual frequency spacing. Log compression handles loudness perception. The DCT handles feature correlation. This is not a black box — every transformation is motivated by either acoustics or statistics.

Normalization is not optional — it directly determines whether gradient descent converges.

What normalization does and why it matters:

The problem with raw pixel values

RGB pixels range from 0 to 255. When you feed these directly into a network, the weight gradients are dominated by large-magnitude inputs. A pixel value of 200 will push gradients much harder than a pixel value of 10, even if they represent equally important visual features.

What min-max normalization does

Dividing every pixel by 255.0 maps all values to the range [0, 1]. This equalises the contribution of every pixel to the gradient updates. The formula is simply: x_norm = x / 255.0. Fast, effective, and standard for most vision tasks.

What standardisation (z-score) does

For tasks requiring tighter control, we subtract the dataset mean and divide by the standard deviation: $x_{norm} = (x - \mu) / \sigma$. This centres the distribution at zero with unit variance, making the loss landscape more isotropic. ImageNet's published mean [0.485, 0.456, 0.406] and std [0.229, 0.224, 0.225] are used across virtually all ImageNet-pretrained models.

Why it changes the loss surface geometry

Unnormalized inputs create elongated, canyon-shaped loss surfaces where gradient descent zigzags inefficiently. Normalized inputs create rounder, more symmetric loss surfaces where gradient descent converges directly. This is not a minor speedup — it can be the difference between a model that trains in 30 epochs and one that needs 300.

SPEAKER NOTES

Every major deep learning framework — PyTorch, TensorFlow, JAX — applies normalization automatically when you use their standard data loaders. But you should understand what they are doing and why. If you ever load data manually and forget to normalize, your first clue will be that the loss refuses to decrease.

Data augmentation is a direct attack on the generalization gap, not a data collection shortcut.

Understanding why augmentation works:

> The generalization gap is the difference between training accuracy and test accuracy. It exists because the model has memorized specific pixel patterns from the training set, not the underlying concept.

The core insight:

If a cat is a cat regardless of whether it is flipped horizontally, rotated 15 degrees, or photographed in dim light — then the model should learn a representation that is invariant to those transformations. Augmentation enforces this invariance during training by making sure the model sees all those variants and receives the same label for each one.

Common augmentations and what invariance each enforces:

- **Horizontal flip:** Teaches the model that left-right orientation is not diagnostic. A dog facing left is still a dog.
- **Random rotation (±15°):** Teaches orientation invariance. Objects do not always appear perfectly upright in photos.
- **Colour jitter (brightness, contrast, saturation):** Teaches appearance invariance across lighting conditions and camera sensors.
- **Random crop and resize:** Teaches that the object of interest is not always centred and full-frame in the image.
- **Gaussian blur / noise:** Teaches robustness to image quality degradation — blurry or noisy images in the real world.
- **Cutout / random erasing:** Teaches the model to rely on the full object, not just one distinctive region. Forces global representation learning.

SPEAKER NOTES

The key word is 'invariance.' When you apply horizontal flipping, you are making an explicit claim: the label of this image does not depend on left-right orientation. That is domain knowledge — you are encoding what you know about the problem directly into the training process. This is why augmentation choices should be deliberate, not random.

Augmentation choices must reflect the semantics of your problem — wrong augmentations actively hurt performance.

Not all augmentations are appropriate for all tasks. Applying the wrong transformation tells the model a false invariance, which corrupts the learned representation.

Examples of task-specific augmentation constraints:

✗ **Digit recognition (MNIST, SVHN)**

Do NOT apply vertical flip — a '6' becomes a '9'. Do NOT apply large rotations — a '6' becomes a '9' again at 180°. Horizontal flip is also problematic for '3', '4', '7'. This is a domain where augmentation must be heavily restricted.

✗ **Medical imaging (chest X-rays, pathology slides)**

Do NOT flip left-right — in cardiology, the heart position relative to the left/right of the body is diagnostically meaningful. Flipping would produce anatomically impossible images. Augmentations must be validated by a medical expert before use.

✗ **Document or text images (OCR)**

Do NOT apply aggressive perspective transforms or rotations beyond a few degrees — text reading direction is semantically critical. Augmentation here is mostly limited to brightness/contrast adjustments and small elastic distortions.

✓ **Satellite / aerial imagery**

This is the opposite case — all rotations including 90° and 180° ARE valid because aerial photographs have no canonical 'up'. Vertical flip is also valid. Aggressive augmentation is

SPEAKER NOTES

This is where you demonstrate expertise as a practitioner. Anyone can copy-paste a standard augmentation pipeline. The mark of an experienced engineer is knowing when NOT to apply a transformation. Always ask: does this augmentation preserve the label? If there is any doubt, do not use it.

Preprocessing is feature engineering — and it is where your domain knowledge enters the model.

Every preprocessing decision is a hypothesis about what structure in the data matters for the prediction task. The model learns to exploit whatever structure you present to it.

Unpacking what this means:

Mel filterbanks encode a theory of auditory perception

When you apply mel-scale frequency spacing, you are asserting: 'The distinction between 200 Hz and 300 Hz matters more than the distinction between 8000 Hz and 8100 Hz.' This is a domain hypothesis. If it is wrong, MFCCs will hurt you. It is correct for speech. It may not be correct for music transcription or bat echolocation.

Horizontal flip encodes a theory of visual symmetry

Applying horizontal flip asserts: 'Left-right orientation is not predictive of the class label.' For natural image classification this is mostly true. For medical imaging it is often dangerously false. Your augmentation strategy is a formal commitment to a set of invariances.

Normalization encodes a theory about the input distribution

Using ImageNet mean/std when fine-tuning on a medical dataset asserts that your medical images have a similar distribution to ImageNet. This is often wrong. The correct practice is to compute mean/std on your own training set. Using the wrong statistics will bias every activation in the network.

SPEAKER NOTES

This framing — preprocessing as hypothesis encoding — is the most important conceptual shift in this module. It moves preprocessing from being a chore into being a design decision. Every time you choose a transformation, you should be able to articulate the hypothesis it embodies. If you cannot, you should not apply it.

Improving your preprocessing pipeline is almost always a better investment than making your model deeper.

The most consistent finding across deep learning research: when a model underperforms, the problem is in the data pipeline more often than in the architecture.

Why this is consistently true:

1. **More parameters do not help if the signal is not there**

   Doubling the depth of a network doubles the number of learnable weights. But if the training data is not normalized, the gradient updates are still distorted — the extra parameters just learn distorted features with more precision. You have more model capacity pointing in the wrong direction.

2. **Augmentation is effectively free compute**

   A horizontal flip augmentation is one line of code and takes microseconds per image. It is equivalent to doubling your training set for the specific invariance it encodes. No architecture change gives you that ratio of effort to improvement.

3. **Preprocessing errors compound through the network**

   A normalization error at the input layer propagates through every subsequent layer. The network will compensate in its weights, but inefficiently. The deeper your network, the more damage a single preprocessing mistake causes.

4. **Transfer learning requires preprocessing to match**

   When you use a pretrained ResNet or ViT, you are loading weights that were optimised on data preprocessed a specific way. If your input pipeline produces different statistics, the pretrained representations immediately lose their meaning. The first step of any fine-tuning task should be verifying that your preprocessing matches the original training conditions.

SPEAKER NOTES

I want you to leave this module with a specific reflex. When your model is not performing as expected, the first question is not 'should I add more layers?' — it is 'is my data pipeline correct?' Check your normalization statistics. Check that augmentations are label-preserving. Check that your preprocessing matches whatever pretrained model you are using. Nine times out of ten, that is where the problem lives.

Preprocessing is the foundation — not the preamble.

1

Raw data is never model-ready

Every input modality — audio, image, text, tabular — requires transformation before a neural network can learn from it effectively. This is not optional.

2

Audio preprocessing compresses and restructures

MFCCs reduce 44,100 samples/second to 13–40 perceptually-meaningful coefficients per frame, mirroring the compression the human cochlea performs automatically.

3

Image preprocessing normalizes and augments

Normalization fixes the loss surface geometry. Augmentation encodes invariances that reduce the generalization gap. Both are essential for any serious vision task.

4

Every preprocessing choice is a domain hypothesis

When you choose a transformation, you are asserting a belief about what structure matters. The best practitioners can articulate the hypothesis behind every step in their pipeline.

5

Better preprocessing beats a bigger model

When performance is insufficient, fix the pipeline first. More parameters amplify whatever signal is present — if the signal is corrupted, more depth makes things worse, not better.

"Your preprocessing choices are hypotheses about what structure matters in your data. Choose deliberately."